Spring 2026 Capstone

Project Assignment

Building a Movie Recommender System

Collaborative Filtering, Content-Based Enrichment,

and Hybrid Recommendation via Neo4j GDS

Skills Practiced: Graph construction from CSV, exploratory Cypher queries, GDS projections, node similarity (Jaccard & cosine), k-nearest neighbors, FastRP embeddings, community detection, collaborative filtering, content-based recommendation, hybrid scoring

Estimated Time: 8–12 hours across multiple work sessions

Group Policy: You may work in small groups of no more than three (3) people for this project

Overview

For this project, you will build and evaluate a **collaborative filtering recommender system** for movies using Neo4j Graph Data Science (GDS). You will start from raw CSV data files, construct a graph database that captures users, their demographic profiles, and movie metadata, and then apply a progression of graph algorithms to generate personalised recommendations for any user in the network.

Recommender systems are among the most commercially significant applications of graph analytics. Streaming platforms, e-commerce marketplaces, and social media feeds all rely on graph-based recommendation engines to surface relevant content. The collaborative filtering paradigm—the idea that users who agreed in the past will agree in the future—maps naturally onto a bipartite graph of users and items. Neo4j's GDS library provides the algorithmic building blocks (node similarity, k-nearest neighbors, FastRP embeddings, community detection) to implement this paradigm at scale.

The dataset provided for this project contains 20 users, 25 movies, and 100 ratings. Although the dataset is compact, it is rich enough to demonstrate the full recommender pipeline: users have demographic attributes (age, gender, occupation), movies carry dual-genre classifications and director metadata, and the rating distribution reflects realistic patterns including popularity bias and rating inflation. Your task is to combine Cypher-based exploration, GDS-powered similarity computation, and thoughtful recommendation logic to produce a system whose outputs you can inspect, evaluate, and defend.

This project assumes you are comfortable writing intermediate-to-advanced Cypher queries, creating GDS graph projections, and interpreting algorithm output in context. Where the assignment leaves room for methodological choice, you are expected to exercise professional judgment, justify your decisions in writing, and demonstrate the analytical maturity appropriate to a graduate-level data scientist.

Files Provided

Three CSV files are provided via the Canvas page for this project. Download all three to a location accessible by your Neo4j instance.

File	Description
users.csv	20 users with demographic attributes: userId, name, age, gender, and occupation
movies.csv	25 movies with metadata: movieId, title, year, genre1, genre2, director, and avgRating
ratings.csv	100 user–movie ratings on a 1–5 scale with timestamps, forming the primary interaction graph

Each movie has up to two genre classifications (genre1, genre2) reflecting multi-genre tagging. The avgRating field represents a pre-computed average rating for each movie on a scale that is **independent** of the user ratings in ratings.csv—it can be treated as an external quality signal (e.g., an aggregated critic or community consensus score). This distinction between the per-user ratings and the movie-level avgRating is important for several of the analytical questions that follow.

Step 1 — Load the Data into Neo4j

Before you can analyze the graph, you must construct it. This step walks you through importing the CSV files into a Neo4j database. All Cypher code in this section is provided for you; your task is to execute it, verify the results, and understand what each statement does.

1.1 Create Uniqueness Constraints

Constraints ensure data integrity and improve MATCH performance during the import process.

```
CREATE CONSTRAINT user_id IF NOT EXISTS
  FOR (u:User) REQUIRE u.userId IS UNIQUE;

CREATE CONSTRAINT movie_id IF NOT EXISTS
  FOR (m:Movie) REQUIRE m.movieId IS UNIQUE;
```

1.2 Load Users

```
LOAD CSV WITH HEADERS FROM 'file:///users.csv' AS row
```

```
CREATE (u:User {
  userId:     row.userId,
  name:       row.name,
  age:        toInteger(row.age),
  gender:     row.gender,
  occupation: row.occupation
});
```

1.3 Load Movies

```
LOAD CSV WITH HEADERS FROM 'file:///movies.csv' AS row
CREATE (m:Movie {
  movieId:   row.movieId,
  title:     row.title,
  year:      toInteger(row.year),
  genre1:    row.genre1,
  genre2:    row.genre2,
  director:  row.director,
  avgRating: toFloat(row.avgRating)
});
```

1.4 Load Ratings as Relationships

```
LOAD CSV WITH HEADERS FROM 'file:///ratings.csv' AS row
MATCH (u:User {userId: row.userId})
MATCH (m:Movie {movieId: row.movieId})
CREATE (u)-[:RATED {
  rating:    toFloat(row.rating),
  timestamp: row.timestamp
}]->(m);
```

1.5 Verify the Graph

```
MATCH (u:User)-[r:RATED]->(m:Movie)
RETURN count(DISTINCT u) AS users,
       count(DISTINCT m) AS movies,
       count(r) AS ratings;
```

Expected result: 20 users, 25 movies, and 100 ratings.

Step 2 — Enrich the Graph with Content Nodes

The raw CSV import creates a simple bipartite graph of Users and Movies connected by RATED relationships. For content-based and hybrid recommendations, the graph needs additional structure. In this step you will extract Genre and Director entities from Movie properties and materialise them as first-class nodes with their own relationships.

2.1 Create Genre Nodes

Each movie carries up to two genre tags. The following Cypher statements create a Genre node for each unique genre and link movies to their genres via IN_GENRE relationships.

```
// Primary genre (always present)
MATCH (m:Movie) WHERE m.genre1 IS NOT NULL
MERGE (g:Genre {name: m.genre1})
MERGE (m)-[:IN_GENRE]->(g);

// Secondary genre (may be null)
MATCH (m:Movie) WHERE m.genre2 IS NOT NULL
MERGE (g:Genre {name: m.genre2})
MERGE (m)-[:IN_GENRE]->(g);
```

2.2 Create Director Nodes

```
MATCH (m:Movie) WHERE m.director IS NOT NULL
MERGE (d:Director {name: m.director})
MERGE (m)-[:DIRECTED_BY]->(d);
```

2.3 Verify Enrichment

```
CALL db.labels() YIELD label
CALL db.stats.retrieve('GRAPH COUNTS') YIELD nodeCount
RETURN label, nodeCount;
```

After enrichment, your graph should contain four node labels (User, Movie, Genre, Director) and three relationship types (RATED, IN_GENRE, DIRECTED_BY). Confirm these counts before proceeding.

Exploratory Data Analysis

With the graph built and enriched, your first analytical task is exploration. The queries below are designed to build your intuition about the data's shape, distributions, and potential biases—all of which will influence how you design your recommender later.

For each query, provide: (a) your Cypher code, (b) the result output, and (c) a brief narrative explaining what the result reveals about the data and how it might affect recommendation quality.

EDA Queries (20 points)

1) **(2 pts)** Write and execute a Cypher query that returns the total number of nodes and relationships in the graph. Include a breakdown by node label and relationship type.
2) **(2 pts)** Write and execute a Cypher query that identifies the five most active raters in the network (users who have rated the most movies). For each, include the user's name, occupation, and the number of movies rated.
3) **(3 pts)** Write and execute a Cypher query that returns the ten most-rated movies. For each movie, include its title, the count of user ratings it has received, the average user rating (computed from RATED relationships), and the movie's avgRating property. Comment on whether popular movies tend to receive higher or lower user ratings compared to their avgRating values.

4) **(3 pts)** Write and execute a Cypher query that computes the rating distribution across the entire dataset: how many ratings of 1, 2, 3, 4, and 5 exist? Comment on whether the distribution is symmetric or skewed and what implications this has for collaborative filtering.

5) **(3 pts)** Write and execute a Cypher query that lists all distinct genres and, for each genre, the number of movies tagged with that genre and the average user rating of movies in that genre. Present results sorted by average rating descending.

6) **(3 pts)** Write and execute a Cypher query that identifies which directors have the most movies in the dataset and, for each, the average user rating across their filmography. List the top 10 directors.

7) **(2 pts)** Write and execute a Cypher query that computes the average number of ratings per user, the median, and the standard deviation. Comment on whether the user activity distribution appears uniform or follows a power-law pattern.

8) **(2 pts)** Write and execute a Cypher query that identifies "polarising" movies: movies with at least 3 ratings where the standard deviation of ratings is highest. List the top 5. Discuss why polarising movies may cause difficulty for collaborative filtering.

Deeper Analytical Questions

Now that you have developed a thorough understanding of the data's characteristics, answer the following analytical questions using Cypher queries. For each question, provide your Cypher query, a written explanation of the query logic, and commentary that explains any conclusions you are able to derive from the results.

Analytical Queries Pick any two (2) queries (30 points)

1) **(15 pts) Taste Overlap Without Algorithms.** Before applying GDS, use pure Cypher to find users who share the most co-rated movies (i.e., pairs of users who have both rated the same movie). For the top 10 user pairs by co-rated movie count, also compute the average absolute difference between their ratings on those shared movies. Discuss whether a high overlap count reliably indicates similar taste, or whether the rating differences tell a more nuanced story.

2) **(15 pts) Genre Preference Profiles.** For each user, compute a "genre preference profile": the average rating the user gives to movies in each genre. Then identify the three users whose genre profiles are most similar to the user 'Alice Chen' (or another user of your choice). You may measure similarity using Euclidean distance, cosine similarity on the profile vectors, or another metric of your choosing. Explain your

similarity measure and discuss whether genre-level profiles are a useful signal for recommendation.

3) **(15 pts) The Long-Tail Problem.** Calculate the number of ratings received for each movie. Then, create a cumulative distribution of ratings across movies: Identify the percentage of ratings represented by the three movies that have received the largest quantity of ratings. Identify at least five movies that are rated highly (average rating ≥ 4.0) but have received very few ratings (bottom quartile of rating count). These are "hidden gems." Discuss the challenge of recommending long-tail items and propose at least one strategy for surfacing them within a graph-based recommender.

4) **(15 pts) Director and Genre Co-Occurrence.** Using the enriched graph (Genre and Director nodes), identify which directors are most strongly associated with specific genres. Then find "crossover directors" who have directed movies spanning two or more distinct genres. For each crossover director, list their genre distribution. Discuss how director–genre relationships could be leveraged as a content-based signal in a hybrid recommender.

Graph Data Science — Building the Recommender

The questions in this section require the use of Neo4j's Graph Data Science library. They follow the natural arc of building a recommender system: first compute user similarity, then generate recommendations, then enrich with content-based signals, and finally evaluate what you've built.

For each question, you must:

- **Create an appropriate in-memory graph projection.** Provide the Cypher code used to create the projection and a written explanation of your design choices (which nodes, which relationships, which properties, directed vs. undirected, weighted vs. unweighted).
- **Select and apply a GDS algorithm.** Provide the Cypher/GDS code, explain why you chose that specific algorithm over alternatives, and connect your rationale to the analytical question being asked.
- **Interpret the results.** Provide commentary that derives actionable or informative conclusions from the algorithm's output. Where appropriate, visualise or tabulate results for clarity.

GDS Analysis Tasks (50 points)

1) **(10 pts) User Similarity via Node Similarity (Jaccard).** Project the bipartite User–Movie rating graph into GDS memory. Run the Node Similarity algorithm in stream mode to compute Jaccard similarity between all user pairs, returning the top 5 neighbors for each user. Present the 15 most similar user pairs. Then write the similarity relationships back to the graph as SIMILAR_TASTE edges with a score property. Discuss what Jaccard similarity captures in this context (shared movie overlap, ignoring rating magnitude) and when this is sufficient versus when it falls short.

2) **(12 pts) User Similarity via k-Nearest Neighbors with FastRP Embeddings.** Create a second GDS projection that includes rating weights. Use the FastRP algorithm to generate low-dimensional embeddings for each user, capturing their weighted rating neighborhood. Then run the kNN algorithm on these embeddings to compute cosine-based user similarity. Present the top 15 user pairs by cosine similarity. Write the results back as KNN_SIMILAR edges. Compare the kNN results with the Jaccard results from the previous task: do the same user pairs appear? Where do the rankings diverge, and what explains the difference? Discuss the trade-offs between unweighted (Jaccard) and weighted (cosine on embeddings) similarity for this dataset.

3) **(10 pts) Generating Personalised Recommendations.** Using the similarity relationships you created in the previous two tasks (SIMILAR_TASTE and/or KNN_SIMILAR), write a Cypher recommendation query that, for a given target user: (a) finds their most similar users, (b) identifies movies those similar users rated highly (rating ≥ 4), (c) excludes movies the target user has already rated, and (d) ranks candidates by the number of supporting similar users and average rating. Generate and present the top 10 recommendations for at least three different users. For each user, briefly assess whether the recommendations appear reasonable given the user's known rating history.

4) **(10 pts) Hybrid Recommendation — Content-Based Enrichment.** Design and implement a hybrid recommendation query that combines collaborative filtering signals (from the similarity-based approach above) with content-based signals (genre and director overlap). Specifically, for a target user, the query should boost candidate movies that share a genre or director with movies the user has already rated ≥ 4. Present the hybrid top 10 recommendations for the same three users you used in the previous task. Compare the hybrid results with the pure collaborative filtering results. Discuss whether the content-based signals changed the rankings, and if so, in what direction and why.

5) **(8 pts) Community Detection and Recommendation Evaluation.** Apply a community detection algorithm (Louvain, Label Propagation, or another algorithm of

your choice) to the user–user similarity graph to identify clusters of users with shared taste. For the three largest communities, characterize their members by dominant genre preferences, average age, and occupation distribution. Then, perform a simple hold-out evaluation: for a user of your choice, temporarily remove two of their ratings, regenerate recommendations, and check whether the held-out movies appear in the top K, where you can choose a value for K between 3 and 10. Report the precision@ K for this experiment. Discuss the limitations of a single-user hold-out test and what a production evaluation pipeline would look like.

Extension Exercises

The following exercises extend the core project. They are designed to deepen your understanding of the recommender system's behaviour and limitations. Complete all exercises.

Exercises (included in GDS section grading) 10 points each

1) **Similarity Cutoff Sensitivity.** Re-run the Node Similarity algorithm with similarityCutoff values of 0.1, 0.3, and 0.5. For each cutoff, report the number of SIMILAR_TASTE relationships created. Then regenerate recommendations for a single user under each cutoff. How does tightening the cutoff affect the number of recommendations returned and their quality? At what point does the graph become too sparse to generate useful recommendations?

2) **Cold-Start Problem.** Create a new user (U021, 'New User') who has rated only one movie (rating of 5). Attempt to generate recommendations using both the collaborative filtering query and the hybrid query. What happens? Which approach degrades more gracefully? Propose and implement at least one concrete strategy for handling cold-start users in this graph (e.g., popularity-based fallback, content-based fallback using the single rated movie's genre and director, demographic-based initialisation). Present the cold-start recommendations your strategy produces.

3) **Algorithm Comparison.** Generate the top 5 recommendations for a single user using three different approaches: (a) Jaccard-based collaborative filtering, (b) kNN/cosine-based collaborative filtering, and (c) your hybrid query. Present the three result sets side by side. Where do they agree? Where do they diverge? Which approach would you deploy in a production system, and why?

4) **Director Affinity Boost.** Modify the hybrid recommendation query to apply an explicit scoring bonus for movies directed by directors whose other films the target user rated ≥ 4. For a user who has rated multiple films by the same director highly, does the

boost surface additional films by that director? Present before and after results and discuss whether affinity boosting improves or distorts recommendation quality.

Deliverable Requirements

Your deliverable for this project is a Jupyter Notebook. It should contain a combination of Cypher commands and queries (executed via the Neo4j Python driver or a Jupyter-compatible Neo4j kernel), the output of those queries, and explanatory narratives contained within properly formatted Markdown cells.

Content Standards

- All Cypher commands should be placed within executable code cells. Query output should be clearly displayed, either rendered natively or incorporated via formatted tables or dataframes.
- Each section of the assignment must begin with a clearly labeled Markdown header that corresponds to the assignment structure.
- Written explanations, query logic descriptions, and analytical commentary should appear in Markdown cells adjacent to the relevant code. Do not rely on inline code comments as a substitute for narrative explanation.
- GDS sections must include: the graph projection code, a rationale for the projection design, the algorithm invocation code, a justification for the algorithm choice, and a substantive interpretation of results.
- The recommendation evaluation section must clearly show the held-out ratings, the generated recommendations, and the precision calculation with supporting reasoning.

Quality Standards

Your Jupyter Notebook deliverable should be similar to that of a publication-quality, professional-caliber document. Specifically:

- Include clearly labeled graphics and visualisations where they strengthen your analysis. At minimum, the rating distribution, community detection clusters, and algorithm comparison results should include visual representations.
- Use high-quality formatting throughout: consistent use of Markdown headers, clean table rendering, professional chart styling, and descriptive captions on all figures and tables.
- Ensure the document is free of spelling and grammatical errors.

- Code cells should be clean, well-commented, and free of debugging artifacts.

Submission Instructions

Upload and submit your Jupyter Notebook within the provided Canvas submission portal. Save your notebook using the following nomenclature: `first-initial_last-name_recommender_project` (e.g., `J_Smith_recommender_project.ipynb`). Small groups should identify all group members at the start of the Jupyter Notebook, and each team member should submit their own copy of the team's work within Canvas.

Grading Summary

Section	Points	Weight
Exploratory Data Analysis (Cypher EDA)	20	20%
Deeper Analytical Questions (Cypher)	30	30%
Graph Data Science + Extensions (GDS)	50	50%
Total	**100**	**100%**

Within each section, points are allocated based on the correctness of your Cypher and GDS code, the depth and clarity of your written explanations, the quality of your analytical conclusions, and overall presentation quality. Queries that return correct results but lack explanatory commentary will receive partial credit. Commentary that demonstrates analytical maturity, domain reasoning, and awareness of recommender-system-specific concerns (sparsity, cold start, popularity bias, evaluation methodology) will receive the highest marks.

Hints and Guidance

The following notes are provided to help you navigate common challenges. They are intentionally non-prescriptive—part of the learning objective is making and defending your own methodological choices.

On Graph Projections

Different GDS tasks may require different projections. A projection for Node Similarity (which operates on the bipartite User–Movie graph) looks very different from one for community detection on a User–User similarity graph. Think carefully about which nodes, relationships, and properties belong in each projection. Overly broad projections introduce noise; overly narrow ones lose signal.

On the Bipartite Structure

The User–Movie rating graph is bipartite: users connect to movies, but users do not directly connect to other users through rating relationships. Node Similarity and kNN handle this naturally by computing similarity through shared neighbours. However, community detection algorithms typically expect a monopartite graph. You will need to reason about how to create a user–user graph from the bipartite structure—the SIMILAR_TASTE or KNN_SIMILAR relationships you computed earlier are a natural starting point.

On Jaccard vs. Cosine Similarity

Jaccard similarity treats ratings as binary signals: either a user rated a movie or they did not. It ignores whether the user gave a 1 or a 5. Cosine similarity (via FastRP embeddings or direct vector comparison) incorporates rating magnitude. Both are valid; they capture different aspects of similarity. Your analysis should reflect an understanding of which is more appropriate in which context.

On Recommendation Evaluation

Evaluating recommender systems properly is difficult. The hold-out experiment in this project is a simplified version of a real evaluation pipeline. Be honest in your write-up about the limitations: a single-user test with two held-out items is not statistically meaningful. A production system would use k-fold cross-validation across all users, measure metrics like precision@k, recall@k, NDCG, and compare against baselines such as popularity-based or random recommendations.

On FastRP Parameters

The FastRP algorithm has several tuning parameters, including embeddingDimension and iterationWeights. For this dataset, an embedding dimension of 64 and iteration weights of [0.0, 1.0, 1.0] are reasonable starting points. Setting the first weight to 0.0 prevents the initial random vectors from dominating the final embedding. Setting randomSeed and concurrency ensures reproducible results—always do this in an academic context.

On Cleaning Up GDS Projections

GDS graph projections live in memory. If you need to recreate a projection (e.g., after adjusting parameters), you must first drop the existing one:

```
CALL gds.graph.drop('your-projection-name');
```

Failing to do this will produce a “graph already exists” error. Consider naming your projections descriptively (e.g., ‘user-movie-unweighted’, ‘user-movie-weighted’) to keep track of what each one contains.

Reference Resources

The following resources may be useful as you work through the project:

- Neo4j Graph Data Science documentation: https://neo4j.com/docs/graph-data-science/current/
- Neo4j Cypher Manual: https://neo4j.com/docs/cypher-manual/current/
- GDS Algorithm reference — Similarity: Node Similarity (Jaccard), k-Nearest Neighbors
- GDS Algorithm reference — Embeddings: FastRP (Fast Random Projection)
- GDS Algorithm reference — Community Detection: Louvain, Label Propagation, Weakly Connected Components
- Neo4j Python Driver documentation: https://neo4j.com/docs/python-manual/current/
- Collaborative Filtering overview: Koren, Bell & Volinsky, “Matrix Factorization Techniques for Recommender Systems,” IEEE Computer, 2009
- Evaluation metrics: Herlocker et al., “Evaluating Collaborative Filtering Recommender Systems,” ACM TOIS, 2004

End of Assignment